Adaptive AI Tutor Research Lab

Personalized Learning with Knowledge Tracing, Recommendation Systems, and Reinforcement Learning

---

Current Status	Synthetic-data research prototype
Next Phase	Real educational dataset validation planned as V2
Focus	End-to-end AI/ML system design for adaptive learning

Prepared by: Shahzada Mustafayev
GitHub: github.com/mstfyvshzde/adaptive-ai-tutor-research-lab
Year: 2026
Stack: Python, scikit-learn, Streamlit, Reinforcement Learning, GitHub Actions

Table of Contents

Project Overview

What Is This Project?

This project is an AI/ML tutor prototype that aims to personalize the learning process for students. The system uses student interaction data, including previous answers, accuracy, topic-level performance, and question difficulty, to estimate a student's current learning state. It is not built around a single model; instead, it connects data generation, feature engineering, supervised prediction models, student clustering, recommendation strategies, and reinforcement learning simulation. The main goal is to explore how an AI tutor can answer three key questions: what a student currently knows, where the student is struggling, and what the student should study next.

Why Was It Built?

This project was built to address a common problem in traditional online learning systems: many students receive the same learning path even though their needs are different. In real life, students are not at the same level. Some students learn quickly, while others need more practice, feedback, or additional support. For this reason, a more effective learning system should analyze a student's previous performance and recommend a more suitable next step. This project explores how AI/ML methods can make the learning process more personalized, adaptive, and measurable.

Current Project Status

This project has not been tested on a real educational dataset yet. The current version is a research-style prototype developed using synthetic student interaction data. The main goal of V1 is to demonstrate whether the full pipeline works end to end, including data generation, feature engineering, supervised learning, clustering, recommendation, reinforcement learning, evaluation, and demo development. Therefore, V1 should be understood as an architecture proof-of-concept rather than a production-ready system. In the next stage, V2 will test the system on real educational datasets such as ASSISTments or EdNet.

Problem and Motivation

The Problem

In general, traditional online learning platforms do not fully consider students' different learning needs and ability levels. Many systems provide the same learning path without carefully analyzing students' previous performance, weak topics, or learning speed. As a result, the content may be too easy for some students while being too difficult for others. This creates a gap between the learning material and the student's actual level.

Motivation

The motivation behind this project is to make learning systems more personalized, adaptive, and data driven. If a system can analyze students' previous answers, accuracy, and topic-level performance, it can better understand what each student actually needs. Instead of providing random or generic content, the system can recommend a more suitable next step based on the student's learning history. This project explores how AI/ML methods can support more effective and personalized learning decisions.

System Architecture

This system is designed as an end-to-end AI/ML pipeline, where data moves through multiple stages from generation to evaluation and demonstration. In the first stage, synthetic student interaction data is created and validated. Then, the raw data is converted into features that represent students' previous performance, accuracy, topic-level behavior, and question difficulty. These features are used in supervised learning models, clustering analysis, and recommendation strategies. The reinforcement learning module simulates the tutor's decision-making process by choosing easy, medium, or hard questions. In the final stage, the results are presented through tables, graphs, and a Streamlit demo.

Pipeline Flow

Data Generation → Validation → Feature Engineering → Supervised Learning → Clustering → Recommendation → Reinforcement Learning → Evaluation → Demo

Main Components

The system consists of several main modules: a data module, a feature engineering module, a supervised learning module, a clustering module, a recommendation module, a reinforcement learning module, an evaluation module, and a demo module. Each module has a specific role in the pipeline. For example, the data module creates and validates the dataset, while the feature engineering module transforms raw interactions into useful learning features. When these modules are connected and run together, they form a complete adaptive tutoring system rather than a single isolated model.

Dataset and Feature Engineering

Dataset

The current version of this project uses synthetic student interaction data. The dataset represents interactions (actions like viewing, answering, or skipping a prompt) between students and questions. Each interaction includes information such as student ID, question ID, timestamp, topic tag, difficulty level, elapsed time (the duration the student spent working on the question), and correctness label. Synthetic data was used to build the system in a safe, controlled, and reproducible way before moving to real educational datasets. This allowed the full pipeline to be developed and tested without privacy risks or dependency on external student data.

Dataset Snapshot	Value
Students	40
Questions	80
Interactions	1452
Average correctness	0.610
Data type	Synthetic

Why Synthetic Data?

Synthetic data was used because the goal was to build and test the system architecture before working with real student data and potential privacy risks. This data allowed the full pipeline to be tested end to end, including data generation, validation, feature engineering, model training, evaluation, and demo development. However, synthetic data does not fully represent real student behavior. Therefore, V1 should be considered an architecture proof-of-concept. In V2, the system will be tested with real educational datasets.

Feature Engineering

In the feature engineering stage, raw synthetic student interaction data was converted into features that the models could understand. The raw data included information such as which student answered which question, whether the answer was correct or incorrect, the topic tag, difficulty level, and elapsed time. However, raw interaction data alone was not enough to represent a student's learning state. Therefore, new features were created to capture previous performance, such as accuracy so far, previous correctness, rolling accuracy, topic-level

performance, question-level accuracy, and estimated question difficulty. These engineered features were used in supervised learning, clustering, and recommendation stages.

Example Feature	Meaning
student_accuracy_so_far	Student's accuracy before the current question
previous_correct	Whether the previous answer was correct
rolling_accuracy_5	Recent performance over the last five attempts
topic_accuracy_so_far	Performance in a specific topic
question_difficulty_estimate	Estimated difficulty of a question
elapsed_time	Time spent answering the question

Supervised Knowledge Tracing

The supervised learning part of this project aims to predict whether a student will answer a question correctly or incorrectly. This task is related to knowledge tracing because the model uses previous student interactions to estimate the student's current learning state. The engineered features, such as previous accuracy, topic-level performance, rolling accuracy, elapsed time, and question difficulty, were used as inputs for the prediction models. Several models were compared, including Dummy Classifier, Logistic Regression, Random Forest, and Gradient Boosting. The Dummy Classifier was used as a baseline to show whether the machine learning models perform better than a simple majority-class prediction. In the current synthetic dataset, Logistic Regression achieved a ROC-AUC score of 0.9077 (a metric ranging from 0 to 1 that measures how effectively the model distinguishes between correct and incorrect answers) and an accuracy of 0.8282. This suggests that the engineered features contain useful predictive information, although the result should be interpreted within the limits of synthetic data.

Model	Accuracy	ROC-AUC	Purpose
Dummy Classifier	0.5876	0.5000	Baseline
Logistic Regression	0.8282	0.9077	Main interpretable model
Random Forest	0.8282	0.8809	Tree-based comparison
Gradient Boosting	0.8247	0.8880	Boosted model comparison

Interpretation: The supervised models performed better than the dummy baseline, which shows that the feature engineering stage created meaningful signals for prediction. However, since the dataset is synthetic, these results should be treated as prototype-level evidence rather than real-world validation.

Student Clustering

The student clustering part of this project aims to group students based on their learning behavior. Instead of only predicting whether a student will answer correctly, clustering helps identify different types of learners in the dataset. Student-level features such as total attempts, final accuracy, average elapsed time, average difficulty level, topic diversity, recent accuracy, and accuracy change were used for clustering. The project used K-Means clustering to divide students into four groups. The current silhouette score is 0.238 (a metric ranging from -1 to 1 that measures how distinct and well-separated the generated clusters are), which suggests moderate cluster separation in the synthetic dataset. This result is useful for understanding learner patterns, but it should not be interpreted as a final real-world student classification because the data is synthetic.

Item	Value
Number of clusters	4
Silhouette score	0.238
Data level	Student-level features
Purpose	Identify learning behavior groups

Interpretation: Clustering adds an exploratory analysis layer to the tutor system. It helps show that students can be grouped by behavior patterns, such as accuracy, speed, difficulty level, and recent performance. In V2, real educational data would make these clusters more meaningful.

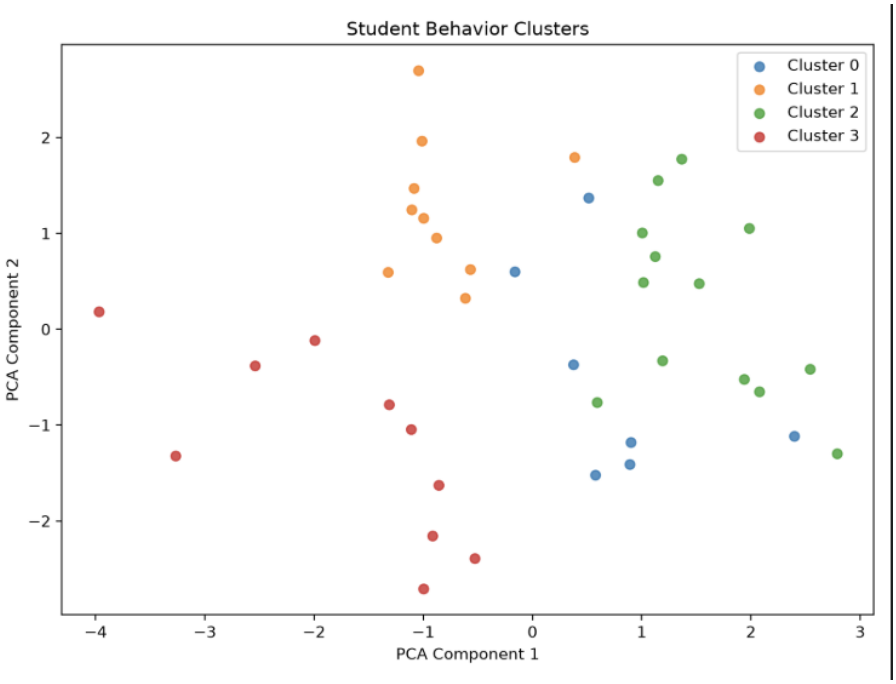


Figure: PCA visualization of student clusters based on learning behavior features.

Recommendation System

The recommendation system part of this project aims to suggest what a student should study or practice next. After the system estimates student performance and learning patterns, recommendation strategies are used to move from prediction to action. The project includes three baseline recommendation approaches: random recommendation, difficulty-based recommendation, and weak-topic recommendation. The random strategy selects a question without personalization. The difficulty-based strategy recommends a question level based on the student's recent accuracy. The weak-topic strategy focuses on the topic where the student has shown lower performance. These recommendation methods are simple, but they demonstrate the core idea of an adaptive tutor: using student data to guide the next learning activity.

Strategy	Purpose
Random recommendation	Selects a question without personalization
Difficulty-based recommendation	Chooses easy, medium, or hard questions based on recent accuracy
Weak-topic recommendation	Focuses on the student's weakest topic

Interpretation: The recommendation system is still a baseline component. Its purpose is not to create a perfect tutor yet, but to show how prediction results and student history can be converted into learning actions. In V2, this part can be improved using real student data and stronger recommendation algorithms.

Reinforcement Learning Tutor

The reinforcement learning part of this project simulates how an adaptive tutor can learn better decision-making over time. In this environment, the tutor observes a simplified student state and chooses an action: giving an easy, medium, or hard question. After each action, the tutor receives a reward (a numerical feedback score indicating the success or correctness of the chosen action) based on the simulated student response. The goal is to learn which difficulty level is more useful for a student's current situation. The project compares a random policy with a Q-learning agent (a model-free algorithm that learns the value of actions by updating its estimations based on the rewards received). In the current synthetic simulation, the Q-learning policy achieved an average reward of 0.762, while the random policy achieved 0.542. This suggests that the learning-based tutor performed better than random decision-making in the prototype environment.

Policy	Average Reward	Average Correctness
Random Policy	0.542	0.562
Q-learning Policy	0.762	0.766

Interpretation: The reinforcement learning module shows how the tutor can move beyond fixed rules and learn from reward signals. However, this result comes from a simplified synthetic environment, so it should be interpreted as a prototype-level simulation rather than real classroom evidence.

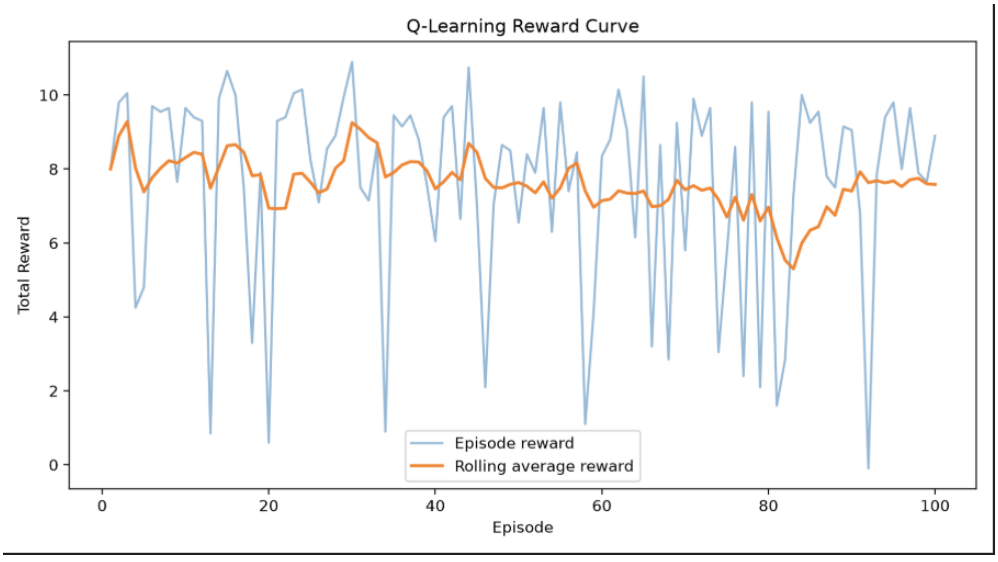


Figure: Q-learning reward curve across training episodes.

Demo and Engineering Quality

This project also includes a Streamlit demo and several engineering-quality components. The demo makes the project easier to understand by presenting the dataset overview, supervised model results, student clustering, recommendation examples, and reinforcement learning comparison in an interactive dashboard. Beyond the demo, the project was designed as a reproducible AI/ML pipeline. The full workflow can be run with python run_pipeline.py, and the repository includes automated testing with pytest, GitHub Actions CI, a Makefile, setup documentation, model cards, methodology notes, and portfolio assets. These components make the project stronger than a simple notebook-based machine learning experiment.

Component	Purpose
Streamlit demo	Presents results visually
run_pipeline.py	Runs the full workflow
pytest	Checks that the project works
GitHub Actions CI	Automates testing on GitHub
Model cards	Documents model purpose and limits
Methodology docs	Explains the research process

Interpretation: The engineering structure shows that the project is not only about training models. It also focuses on reproducibility, documentation, testing, and presentation, which are important parts of professional AI/ML projects.

🧠 Adaptive AI Tutor Research Lab

This project explores how an AI tutor can estimate student knowledge, recommend learning activities, and improve decisions using reinforcement learning.

Students	Questions	Interactions	Average Correctness
40	80	1452	61.02%

1. Student Learning Data

The project uses a synthetic student interaction dataset. Each row represents one student answering one question.

	student_id	question_id	timestamp	part	tags	difficulty	difficulty_score	elapsed_time	student_attempt_count	stuc
0	student_001	70	2026-01-02 11:02:00	1	geometry	hard	0.75	86008	0	
1	student_001	75	2026-01-02 12:15:00	1	functions	medium	0.5	81621	1	
2	student_001	6	2026-01-02 13:46:00	2	probability	easy	0.25	52750	2	
3	student_001	80	2026-01-02 16:22:00	2	statistics	easy	0.25	40543	3	
4	student_001	24	2026-01-02 17:39:00	3	algebra	easy	0.25	35582	4	
5	student_001	61	2026-01-02 20:21:00	2	statistics	hard	0.75	102838	5	
6	student_001	36	2026-01-02 20:40:00	3	statistics	medium	0.5	56670	6	
7	student_001	18	2026-01-02 22:48:00	1	algebra	medium	0.5	75282	7	
8	student_001	6	2026-01-02 23:39:00	2	probability	easy	0.25	69739	8	
9	student_001	60	2026-01-03 00:51:00	2	probability	easy	0.25	66535	9	

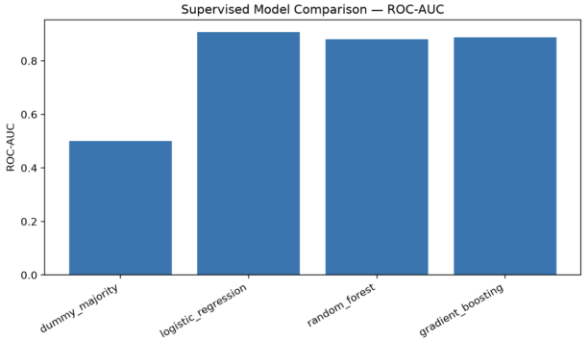
Average Correctness by Topic

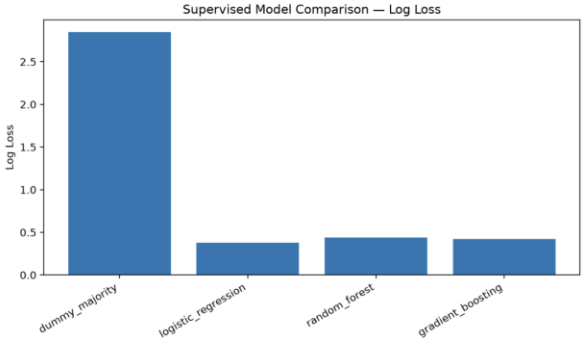
	tags	attempts	average_correctness
0	algebra	355	0.628
1	functions	198	0.682
2	geometry	277	0.477
3	probability	350	0.677
4	statistics	272	0.585

2. Supervised Knowledge Tracing

This section predicts whether a student will answer correctly using features such as previous accuracy, topic history, and question difficulty.

	model	accuracy	precision	recall	f1	roc_auc	log_loss
0	dummy_majority	0.5876	0.5876	1	0.7403	0.5	2.8491
1	logistic_regression	0.8282	0.8457	0.8655	0.8555	0.9077	0.3764
2	random_forest	0.8282	0.8538	0.8538	0.8538	0.8809	0.4383
3	gradient_boosting	0.8247	0.8448	0.8596	0.8522	0.888	0.4206

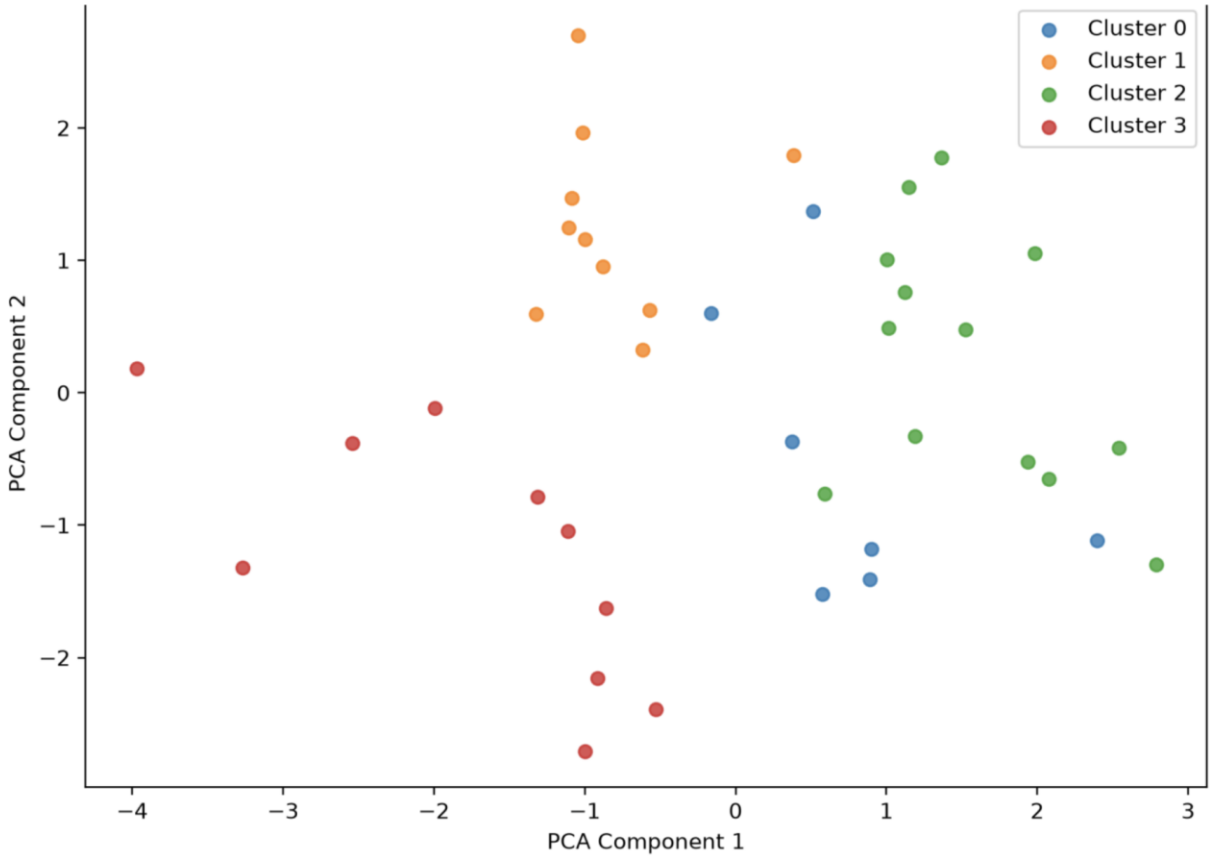




3. Student Clustering

This section groups students based on learning behavior such as accuracy, activity level, topic diversity, and progress.

	cluster	student_total_attempts	student_final_accuracy	student_avg_elapsed_time	student_avg_difficulty_score	student_topic_diversity	student_rece
0	0	42	0.753	62416.425	0.491	5	
1	1	36.8	0.558	69030.619	0.523	5	
2	2	31.692	0.664	63638.726	0.474	5	
3	3	37.8	0.488	66562.113	0.487	5	



4. Recommendation System

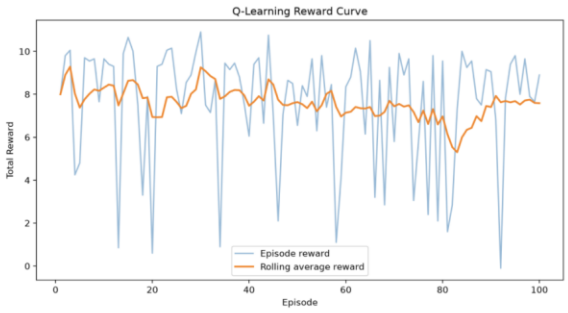
This section shows example recommendations from simple tutor policies: random recommendation, difficulty-based recommendation, and weak-topic recommendation.

	student_id	method	recommended_question_id	recommended_topic	recommended_difficulty
0	student_001	random	48	probability	medium
1	student_001	difficulty_based	38	algebra	hard
2	student_001	weak_topic	12	probability	medium
3	student_002	random	67	algebra	easy
4	student_002	difficulty_based	76	geometry	easy
5	student_002	weak_topic	5	algebra	hard
6	student_003	random	3	algebra	easy

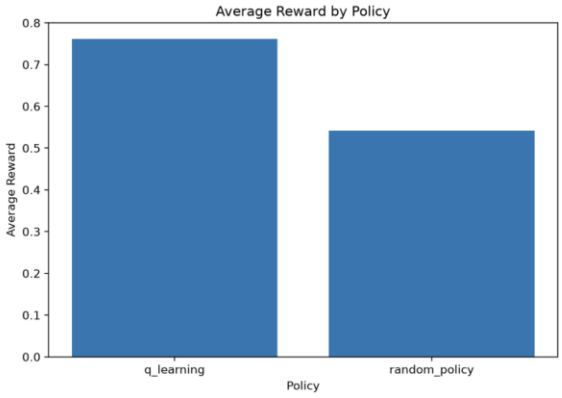
5. Reinforcement Learning Tutor

This section compares a random tutor policy with a Q-learning tutor. The goal is to test whether the tutor can learn better decisions over time.

	policy	average_reward	average_correctness	average_probability_correct	total_steps
0	q_learning	0.762	0.768	0.766	1000
1	random_policy	0.542	0.565	0.562	200



Q-learning reward curve



Limitations

This project has several important limitations. The current version uses synthetic student interaction data, so the results do not prove real-world learning improvement. The student behavior, question difficulty, and reward signals are simulated, which means they may not fully match real educational environments. The recommendation system is also based on simple baseline strategies rather than advanced personalization algorithms. Similarly, the reinforcement learning environment is simplified and does not include long-term classroom feedback, motivation, forgetting, or real student progress over time. Therefore, the current version should be understood as a research-style prototype and architecture proof-of-concept, not as a production-ready educational system.

Limitation	Meaning
Synthetic data	Results are not real-world validation
Simulated student behavior	Real learners may behave differently
Baseline recommender	Recommendation logic is still simple
RL environment	Tutor decisions are tested in simulation
No classroom testing	Learning improvement is not proven yet

Interpretation: These limitations do not make the project weak. They clearly define the boundary of V1 and explain why V2 should focus on real educational datasets and stronger evaluation.

Real Dataset V2 Plan

The next major step of this project is to move from synthetic data to real educational datasets. V2 will focus on testing whether the same adaptive tutoring pipeline can work with real student interaction data. Candidate datasets include ASSISTments, EdNet, or other public educational data sources. In this phase, the main task will be to map real dataset columns into the project structure, including student ID, question ID, timestamp, topic or skill tag, elapsed time, and correctness. After that, the supervised models, clustering analysis, recommendation strategies, and reinforcement learning simulation can be evaluated again. This step will make the project more realistic and closer to research-level validation.

V2 Step	Purpose
Select real dataset	Choose ASSISTments, EdNet, or similar data
Map columns	Convert real data into project format
Rebuild features	Adapt feature engineering to real interactions
Re-run models	Compare real-data results with V1
Improve recommender	Move beyond baseline strategies
Report findings	Add real-data validation to the project

Interpretation: V2 is the research upgrade of this project. V1 proves that the architecture works; V2 will test whether the system remains useful with real educational data.

Conclusion

This project demonstrates how different AI/ML components can be connected to build an adaptive tutoring prototype. Instead of focusing on only one model, the project combines data generation, validation, feature engineering, supervised knowledge tracing, student clustering, recommendation strategies, reinforcement learning simulation, evaluation, visualization, testing, and a Streamlit demo. The current V1 version shows that the full architecture works end to end on synthetic student interaction data. However, the project is not presented as a production-ready educational system or real classroom validation. Its main value is the system design, research-style structure, and reproducible pipeline. The next step is to improve the project with real educational datasets in V2.

Final Summary: V1 proves the architecture. V2 will test the system with real educational data.

Project Links & Reproducibility

GitHub Repository:

github.com/mstfyvshzde/adaptive-ai-tutor-research-lab

Portfolio Website:

shahzada.org

Version:

V1.0 — Synthetic Research Prototype

How to Run the Project

git clone https://github.com/mstfyvshzde/adaptive-ai-tutor-research-lab.git
cd adaptive-ai-tutor-research-lab

python3 -m venv .venv
source .venv/bin/activate

pip install -r requirements.txt
python run_pipeline.py
python -m pytest -q
streamlit run 07_demo/app.py

These commands clone the repository, create a virtual environment, install dependencies, run the full pipeline, execute the smoke test, and launch the Streamlit demo.

Key Results Summary

Area	Key Result
Dataset	40 students, 80 questions, 1452 interactions
Supervised ML	Logistic Regression ROC-AUC 0.9077
Clustering	K-Means with silhouette score 0.238
RL Tutor	Q-learning reward 0.762 vs random 0.542
Engineering	Pipeline, pytest, CI, Streamlit demo

Project Structure

File / Folder	Purpose
run_pipeline.py	Runs the full project
04_src/data/	Creates and validates the synthetic dataset
04_src/features/	Builds learning-history features
04_src/models/	Trains supervised, clustering, and recommender models
04_src/rl/	Simulates tutor decisions with reinforcement learning
07_demo/app.py	Streamlit dashboard
10_docs/	Methodology, setup, ethics, deployment docs